

而不是“继续循环”（想一想，为什么），可以把 `break` 换成 `continue` 加以验证。

另外，注意到这里的 `for` 语句是“残缺”的：没有指定循环条件。事实上，3 部分都是可以省略的。没错，`for(;;)` 就是一个死循环，如果不采取措施（如 `break`），就永远不会结束。

2.2 while 循环和 do-while 循环

例题 2-2 $3n+1$ 问题

猜想^①：对于任意大于 1 的自然数 n ，若 n 为奇数，则将 n 变为 $3n+1$ ，否则变为 n 的一半。经过若干次这样的变换，一定会使 n 变为 1。例如， $3 \rightarrow 10 \rightarrow 5 \rightarrow 16 \rightarrow 8 \rightarrow 4 \rightarrow 2 \rightarrow 1$ 。

输入 n ，输出变换的次数。 $n \leq 10^9$ 。

样例输入：

3

样例输出：

7

【分析】

不难发现，程序完成的工作依然是重复性的：要么乘 3 加 1，要么除以 2，但和 2.1 节的程序又不太一样：循环的次数是不确定的，而且 n 也不是“递增”式的循环。这样的情况很适合用 `while` 循环来实现。

程序 2-4 $3n+1$ 问题（有 bug）

```
#include<stdio.h>
int main()
{
    int n, count = 0;
    scanf("%d", &n);
    while(n > 1)
    {
        if(n % 2 == 1) n = n*3+1;
        else n /= 2;
        count++;
    }
    printf("%d\n", count);
    return 0;
}
```

上面的程序有好几个值得注意的地方。首先是“`=0`”，意思是定义整型变量 `count` 的同时初始化为 0。接下来是 `while` 语句。

^① <http://en.wikipedia.org/wiki/3n+1>。

提示 2-8: while 循环的格式为“while(条件) 循环体;”。

此格式看上去比 for 循环更简单, 可以用 while 改写 for。“for(初始化; 条件; 调整) 循环体;”等价于:

```
初始化;
while(条件)
{
    循环体;
    调整;
}
```

建议读者再次利用 IDE 或者 gdb 跟踪调试, 看看执行流程是怎样的。

$n/=2$ 的含义是 $n=n/2$, 类似于前面介绍过的 $i++$ 。很多运算符都有类似用法, 例如, $a*=3$ 表示 $a=a*3$ 。

$count++$ 的作用是计数器。由于最终输出的是变换的次数, 需要一个变量来完成计数。

提示 2-9: 当需要统计某种事物的个数时, 可以用一个变量来充当计数器。

这个程序是否正确? 先来测试一下: 输入“987654321”, 看看结果是什么。很不幸, 答案等于 1——这明显是错误的。题目中给出的范围是 $n \leq 10^9$, 这个 987654321 是合法的输入数据。

提示 2-10: 不要忘记测试。一个看上去正确的程序可能隐含错误。

问题出在哪里呢? 若反复阅读程序仍然无法找到答案, 就动手实验吧! 一种方法是利用 IDE 和 gdb 跟踪调试, 但这并不是本书所推荐的调试方法。一个更通用的方法是: 输出中间结果。

提示 2-11: 在观察无法找出错误时, 可以用“输出中间结果”的方法查错。

在给 n 做变换的语句后加一条输出语句 `printf("%d\n", n)`, 将很快找到问题的所在: 第一次输出为 -1332004332, 它不大于 1, 所以循环终止。如果认真完成了前面的所有探索实验, 读者将立刻明白这其中的缘由: 乘法溢出了。

下面稍微回顾一下数据类型的大小。在第 1 章中, 通过实验得出了 int 整数的大小——很可能是 -2147483648~2147483647, 即 $-2^{31} \sim 2^{31}-1$ 。为什么叫“很可能”呢, 因为 C99 中只规定了 int 至少是 16 位, 却没有规定具体值^①。是不是感觉有些别扭? 的确如此, 所以 C99 规定了一些固定长度的整数, 例如 `int32_t`、`uint32_t`^②。

好在算法竞赛的平台相对稳定, 目前几乎在所有的比赛平台上, int 都是 32 位整数。

提示 2-12: C99 并没有规定 int 类型的确切大小, 但在当前流行的竞赛平台中, int 都是 32 位整数, 范围是 -2147483648~2147483647。

^① 在笔者中学时期, int 一般是 16 位的, 即 -32768~32767。

^② `uint32_t` 表示无符号 32 位整数, 范围是 0~4294967296。

回到本题。本题中 n 的上限 10^9 只比 `int` 的上界稍微小一点，因此溢出了也并不奇怪。只要使用 C99 中新增的 `long long` 即可解决问题，其范围是 $-2^{63} \sim 2^{63}-1$ ，唯一的区别就是要把输入时的 `%d` 改成 `%lld`。但这也是不保险的——在 MinGW 的 `gcc`^① 中，要把 `%lld` 改成 `%I64d`，但奇怪的是 VC2008 里又得改回 `%lld`。是不是很容易搞错？所以如果涉及 `long long` 的输入输出，常用 C++ 的输入输出流或者自定义的输入输出方法，本书将在后面的章节对其进行深入讨论。

提示 2-13: `long long` 在 Linux 下的输入输出格式符为 `%lld`，但 Windows 平台中有时为 `%I64d`。为保险起见，可以用后面介绍的 C++ 流，或者编写自定义输入输出函数。

最后给出 `long long` 版本的代码，它避开对 `long long` 的输入输出，并且成功算出 $n=987654321$ 时的答案为 180。

程序 2-5 $3n+1$ 问题

```
#include<stdio.h>

int main()
{
    int n2, count = 0;
    scanf("%d", &n2);
    long long n = n2;
    while(n > 1)
    {
        if(n % 2 == 1) n = n*3+1;
        else n /= 2;
        count++;
    }
    printf("%d\n", count);
    return 0;
}
```

例题 2-3 近似计算

计算 $\frac{\pi}{4} = 1 - \frac{1}{3} + \frac{1}{5} - \frac{1}{7} + \dots$ ，直到最后一项小于 10^{-6} 。

【分析】

本题和例题 2-2 一样，也是重复计算，因此可以用循环实现。但不同的是，只有算完一项之后才知道它是否小于 10^{-6} 。也就是说，循环终止判断是在计算之后，而不是计算之前。这样的情况很适合使用 `do-while` 循环。

程序 2-6 近似计算

```
#include<stdio.h>

int main() {
    double sum = 0;
```

^① 这并不是 MinGW 引起的，而是因为 Windows 的 CRT (C Runtime)。